1. Single Inheritance

Problem Statement:

Write a Java program to demonstrate **single inheritance** by creating a `Person` superclass containing the name and age of a person. Create a `Student` subclass containing roll number and branch. Inherit the properties of `Person` and display the complete student details.

Code:

```
// Superclass
class Person {
   String name;
   int age;

   void displayPersonDetails() {
      System.out.println("Name: " + name);
      System.out.println("Age: " + age);
   }
}

// Subclass inherits Person
class Student extends Person {
   int rollNo;
   String branch;

   void displayStudentDetails() {
      displayPersonDetails();
      System.out.println("Roll Number: " + rollNo);
      System.out.println("Branch: " + branch);
   }
}
```

```
// Main class
public class Main {
    public static void main(String[] args) {
        Student s = new Student();


        s.name = "Rahul";
        s.age = 20;
        s.rollNo = 101;
        s.branch = "Computer Science";


        System.out.println("Student Details:");
        s.displayStudentDetails();
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Student Details:
Name: Rahul
Age: 20
Roll Number: 101
Branch: Computer Science

C:\Users\ITLAB2\Desktop\25EU02070>
```

---

2. Using `super` Keyword

Problem Statement:

Write a Java program to demonstrate the use of the `super` **keyword**. Create a `Vehicle` superclass with a speed variable and a `display()` method. Create a `Car` subclass with a model variable and override the `display()` method. Use `super` to access the superclass variable and method.

```
class Vehicle {
    int speed = 80;
```

```
    void display() {
        System.out.println("Vehicle speed: " + speed);
    }
}

class Car extends Vehicle {
    String model = "Toyota";

    @Override
    void display() {
        System.out.println("Car model: " + model);
        System.out.println("Vehicle speed using super: " + super.speed);
        super.display();
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.display();
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Car model: Toyota
Vehicle speed using super: 80
Vehicle speed: 80

C:\Users\ITLAB2\Desktop\25EU02070>
```

3. Multilevel Inheritance

Problem Statement:

Write a Java program to demonstrate **multilevel inheritance** by creating three classes: `Person`, `Employee`, and `Manager`. The `Person` class should contain personal details, `Employee` should contain employee details, and `Manager` should contain department details. Display all the details using inherited properties.

```
class Person {
String name;
int age;

void displayPersonDetails() {
System.out.println("Name: " + name);
System.out.println("Age: " + age);
}
}

class Employee extends Person {
int employeeId;
double salary;

void displayEmployeeDetails() {
System.out.println("Employee ID: " + employeeId);
System.out.println("Salary: " + salary);
}
}

class Manager extends Employee {
String department;

void displayManagerDetails() {
displayPersonDetails();
displayEmployeeDetails();
System.out.println("Department: " + department);
}
}

public class Main {
public static void main(String[] args) {
Manager m = new Manager();

m.name = "Anil";
m.age = 35;
m.employeeId = 1001;
m.salary = 75000;
m.department = "IT";
```

```
m.displayManagerDetails();
}
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>java Main
Name: Anil
Age: 35
Employee ID: 1001
Salary: 75000.0
Department: IT

C:\Users\ITLAB2\Desktop\25EU02070>
```

4. Method Overriding

Problem Statement:

Write a Java program to demonstrate **method overriding** by creating an `Animal` superclass with a `sound()` method. Create subclasses `Dog` and `Cat` that override the `sound()` method with their respective sounds. Display the appropriate sound for each animal.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}


class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}


class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
```

```
    }
}


public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();


        d.sound();
        c.sound();
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Dog barks
Cat meows

C:\Users\ITLAB2\Desktop\25EU02070>
```

5. Dynamic Method Dispatch

Problem Statement:

Write a Java program to demonstrate **dynamic method dispatch** using a superclass `Shape` and subclasses `Circle`, `Rectangle`, and `Triangle`. Override the `draw()` method in each subclass and use a superclass reference to invoke the appropriate overridden method at runtime.

```
class Shape {
void draw() {
System.out.println("Drawing a shape");
}
}

class Circle extends Shape {
@Override
void draw() {
System.out.println("Drawing a circle");
}
}
```

```
class Rectangle extends Shape {
@Override
void draw() {
System.out.println("Drawing a rectangle");
}
}

class Triangle extends Shape {
@Override
void draw() {
System.out.println("Drawing a triangle");
}
}

public class Main {
public static void main(String[] args) {
Shape s;

s = new Circle();
s.draw();

s = new Rectangle();
s.draw();

s = new Triangle();
s.draw();
}
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Drawing a circle
Drawing a rectangle
Drawing a triangle

C:\Users\ITLAB2\Desktop\25EU02070>
```

6. `super` with Method Overriding

Problem Statement:

Write a Java program to demonstrate the use of `super` **with method overriding**. Create an `Employee` class with a `display()` method and derive a `Manager` class that overrides the method. Use `super.display()` in the subclass to invoke the superclass implementation before displaying the manager details.

```
class Employee {
```

```
void display() {
System.out.println("Employee details");
}
}

class Manager extends Employee {
@Override
void display() {
super.display();
System.out.println("Manager details");
}
}

public class Main {
public static void main(String[] args) {
Manager m = new Manager();
m.display();
}
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Employee details
Manager details
```

7. Multilevel Inheritance for Salary Calculation

Problem Statement:

Write a Java program to create a multilevel inheritance hierarchy `Employee → Developer → SeniorDeveloper`. Store basic salary in the `Employee` class, programming allowance in the `Developer` class, and project allowance in the `SeniorDeveloper` class. Calculate and display the total salary.

```
class Employee {
double basicSalary = 30000;

void displayBasicSalary() {
System.out.println("Basic Salary: " + basicSalary);
}
}

class Developer extends Employee {
double programmingAllowance = 10000;

void displayProgrammingAllowance() {
```

```
System.out.println("Programming Allowance: " +
programmingAllowance);
}
}

class SeniorDeveloper extends Developer {
double projectAllowance = 15000;

void calculateSalary() {
double totalSalary = basicSalary
+ programmingAllowance
+ projectAllowance;

displayBasicSalary();
displayProgrammingAllowance();
System.out.println("Project Allowance: " + projectAllowance);
System.out.println("Total Salary: " + totalSalary);
}
}

public class Main {
public static void main(String[] args) {
SeniorDeveloper sd = new SeniorDeveloper();
sd.calculateSalary();
}
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>java Mai
Basic Salary: 30000.0
Programming Allowance: 10000.0
Project Allowance: 15000.0
Total Salary: 55000.0

C:\Users\ITLAB2\Desktop\25EU02070>
```

8. Dynamic Method Dispatch for Bank Accounts

Problem Statement:

Write a Java program to demonstrate **dynamic method dispatch** using a superclass `BankAccount` with a `calculateInterest()` method. Create `SavingsAccount` and `CurrentAccount` subclasses with different implementations of the method. Use a superclass reference to calculate interest for different account types.

```
class BankAccount {
void calculateInterest() {
System.out.println("Calculating bank account interest");
}
}
```

```
class SavingsAccount extends BankAccount {
@Override
void calculateInterest() {
System.out.println("Savings Account Interest: 6%");
}
}

class CurrentAccount extends BankAccount {
@Override
void calculateInterest() {
System.out.println("Current Account Interest: 2%");
}
}

public class Main {
public static void main(String[] args) {
BankAccount account;

account = new SavingsAccount();
account.calculateInterest();

account = new CurrentAccount();
account.calculateInterest();
}
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Savings Account Interest: 6%
Current Account Interest: 2%

C:\Users\ITLAB2\Desktop\25EU02070>
```

9. Constructor Execution in Multilevel Inheritance

Problem Statement:

Write a Java program to demonstrate the **order of constructor execution in multilevel inheritance** using the classes `Person`, `Student`, and `GraduateStudent`. Use constructors in all three classes and display messages to show the order in which the constructors are executed.

```
class Person {
Person() {
System.out.println("Person constructor executed");
}
}
```

```
class Student extends Person {
Student() {
System.out.println("Student constructor executed");
}
}

class GraduateStudent extends Student {
GraduateStudent() {
System.out.println("GraduateStudent constructor executed");
}
}

public class Main {
public static void main(String[] args) {
GraduateStudent gs = new GraduateStudent();
}
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Person constructor executed
Student constructor executed
GraduateStudent constructor executed

C:\Users\ITLAB2\Desktop\25EU02070>
```

10. Banking Application Using Inheritance

Problem Statement:

Write a Java program to develop a simple **banking application using inheritance**. Create a `BankAccount` superclass with account number and balance. Derive `SavingsAccount` and `CurrentAccount` classes and provide suitable methods for deposit, withdrawal, and interest calculation. Demonstrate inheritance and method overriding.

```
class BankAccount {
String accountNumber;
double balance;

BankAccount(String accountNumber, double balance) {
this.accountNumber = accountNumber;
this.balance = balance;
}

void deposit(double amount) {
balance += amount;
System.out.println("Deposited: " + amount);
}
```

```
void withdraw(double amount) {
if (amount <= balance) {
balance -= amount;
System.out.println("Withdrawn: " + amount);
} else {
System.out.println("Insufficient balance");
}
}

void calculateInterest() {
System.out.println("General bank account interest");
}

void displayBalance() {
System.out.println("Account Number: " + accountNumber);
System.out.println("Balance: " + balance);
}
}

class SavingsAccount extends BankAccount {
SavingsAccount(String accountNumber, double balance) {
super(accountNumber, balance);
}

@Override
void calculateInterest() {
double interest = balance * 0.06;
System.out.println("Savings Interest: " + interest);
}
}

class CurrentAccount extends BankAccount {
CurrentAccount(String accountNumber, double balance) {
super(accountNumber, balance);
}

@Override
void calculateInterest() {
double interest = balance * 0.02;
System.out.println("Current Account Interest: " + interest);
}
}

public class Main {
public static void main(String[] args) {
```

```
SavingsAccount savings =
new SavingsAccount("SA101", 10000);

System.out.println("Savings Account:");
savings.deposit(2000);
savings.withdraw(1000);
savings.calculateInterest();
savings.displayBalance();

System.out.println();

CurrentAccount current =
new CurrentAccount("CA101", 20000);

System.out.println("Current Account:");
current.deposit(5000);
current.withdraw(3000);
current.calculateInterest();
current.displayBalance();
}
}
```

```
C:\Users\ITLAB2\Desktop\25EU02070>javac Main.java

C:\Users\ITLAB2\Desktop\25EU02070>java Main
Savings Account:
Deposited: 2000.0
Withdrawn: 1000.0
Savings Interest: 660.0
Account Number: SA101
Balance: 11000.0

Current Account:
Deposited: 5000.0
Withdrawn: 3000.0
Current Account Interest: 440.0
Account Number: CA101
Balance: 22000.0

C:\Users\ITLAB2\Desktop\25EU02070>
```